COMSATS University Islamabad, Vehari Campus, Pakistan

Department of Computer Science

Smart POS System with Google Sheets



A Thesis
for the Degree of

Bachelor of Science in Computer Science

Submitted By

Ghazal Riaz (FA22-BCS-066)

Supervisor

Muhammad Abdullah

Session: Spring 2022-2026

Dedication

I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey.

I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity.

Acknowledgement

All praise and gratitude are due for the successful completion of this project. I would like to express their sincere appreciation to my supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis. The supervision and mentorship provided significant support in completing this work successfully.

Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program.

I also deeply grateful to my parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength.

Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project.

Certificate of Approval

This is to certify that the Final Year Project titled **"Smart POS System with Google Sheets"** has been developed by **Ghazal Riaz (FA22-BCS-066)** under the supervision of **Muhammad Abdullah** and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science.

_____

Supervisor

_____

External Examiner

_____

Head of Department
Department of Computer Science

Plagiarism and AI Content Certificate

Date: _____

It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the **Thesis** submitted by:

- **Ghazal Riaz (FA22-BCS-066)**

titled:

"Smart POS System with Google Sheets"

is **[%]** which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith.

Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as **[%]**. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools.

The report is recommended for submission.

Supervisor Name: _____

Designation: _____

Signature: _____

Department: Department of Computer Science

University: COMSATS University Islamabad, Vehari Campus

Executive Summary

The Smart POS System is a comprehensive, local-first point-of-sale and inventory management solution designed to bridge the digital gap for small-to-medium retail enterprises. Traditional retail operations often suffer from manual calculation errors, data loss, and high costs associated with complex enterprise ERP systems. This project addresses these challenges by providing a robust, high-performance cross-platform application built with the Flutter framework.

By utilizing an embedded SQLite relational database, the system ensures 100% operational autonomy, allowing retailers to process sales, manage stock, and generate reports without a constant internet connection. To ensure data redundancy and remote accessibility, the system integrates the Google Sheets API for cost-free cloud synchronization. Key technical features include a "Premium UX" design with haptic feedback, automated barcode scanning, PDF receipt generation, and a real-time analytics dashboard that provides strategic insights into profit margins and sales trends.

This project delivers a scalable, maintainable, and user-friendly digital ecosystem that professionalizes local business data. The resulting application not only improves transaction efficiency but also empowers business owners with precise financial oversight, ultimately fostering growth through digital transformation.

Abbreviations

Abbreviation	Full Form
SRS	Software Requirements Specification
SDD	Software Design Document
API	Application Programming Interface
UI	User Interface
DB	Database

Table of Contents

Executive Summary v

List of Tables **x**

List of Figures **xi**

1 Introduction **1**
1.1 Introduction 1
1.2 Vision Statement 1
1.3 Related System Analysis 1
1.4 Project Deliverables 2
1.5 System Limitations and Constraints 2
1.6 Tools and Technologies 3
1.7 Relevance to Course Modules 3
1.7.1 Programming Fundamentals 3
1.7.2 Data Structures and Algorithms 3
1.7.3 Database Management Systems 3
1.7.4 Software Engineering 4
1.7.5 Human Computer Interaction (HCI) 4
1.8 Chapter Summary 4

2 Problem Definition **5**
2.1 Problem Statement 5
2.2 Proposed Solution Overview 5
2.3 Objectives of the Proposed System 5
2.4 Scope of the System 6
2.5 System Architecture and Modules 6
2.5.1 Module 1: Authentication & Security 6
2.5.2 Module 2: Inventory & Product Management 7
2.5.3 Module 3: Sales Engine (Core Module) 7
2.5.4 Module 4: Analytics & Reporting (Admin Module) 7
2.5.5 Module 5: Integration & Cloud Service 8
2.6 Assumptions and Dependencies 8
2.7 Chapter Summary 8

3 Requirement Analysis **9**
3.1 Introduction to Requirement Analysis 9
3.2 Requirement Elicitation Techniques 9

3.3 User Classes and Characteristics . . . 10
3.4 Use Case Model . . . 10
3.4.1 Use Case Diagram Description . . . 10
3.4.2 Detailed Use Case Specifications . . . 10
3.4.2.1 UC-1.1: Intelligent Sales Processing . . . 11
3.4.2.2 UC-1.2: Cloud Synchronization and Backup . . . 11
3.5 Functional Requirements . . . 11
3.5.1 Module 1: Authentication and Audit Systems . . . 11
3.5.2 Module 2: Inventory and Multi-Supplier Management . . . 12
3.5.3 Module 3: Sales Engine and Promotions . . . 12
3.6 Non-Functional Requirements . . . 12
3.6.1 Performance Requirements . . . 12
3.6.2 Security and Data Integrity . . . 13
3.6.3 Availability and Connectivity . . . 13
3.7 External Interface Requirements . . . 13
3.7.1 Users Interface Requirement . . . 13
3.7.2 Hardware Interface Requirements . . . 13
3.8 Requirement Traceability Matrix . . . 14

4 Software Design **15**
4.1 Introduction to Software Design . . . 15
4.2 Design Methodology and Software Process Model . . . 15
4.2.1 Design Methodologys . . . 15
4.2.2 Software Processes Model . . . 15
4.3 System Overview . . . 16
4.4 Architectural Design . . . 16
4.5 Design Models . . . 18
4.5.1 Activity Diagrams . . . 18
4.5.1.1 Module 1: Sales Engine Workflow . . . 18
4.5.2 Class Diagram . . . 19
4.5.3 Sequence Diagrams . . . 19
4.5.3.1 Sequence Diagram – Processing a Sale . . . 20
4.5.3.2 Sequence Diagram – Google Sheets Synchronisation . . . 20
4.5.4 State Transition Diagram . . . 21
4.6 Data Flow Diagrams . . . 22
4.6.1 Level 0 Data Flow Diagram (Context DFD) . . . 22
4.6.2 Level 1 Data Flow Diagram . . . 23
4.7 Data Design . . . 24
4.7.1 Entity Relationship Diagram (ERD) . . . 24
4.8 Database Schema Diagram . . . 25
4.8.1 Data Dictionary . . . 26
4.9 Chapter Summary . . . 27

5 Implementation **28**
5.1 Development Environment . . . 28
5.2 Core Module Implementations . . . 28

5.2.1 Module 1: Reactive Sales Engine . . . 28
5.2.2 Module 2: Structured Inventory Management . . . 28
5.3 Algorithm Implementation . . . 29
5.3.1 Algorithm 1: Real time Cart Aggregation . . . 29
5.3.2 Algorithm 2: Inventory Deduction and Low-Stock Alert . . . 29
5.3.3 Algorithm 3: Cloud Sync Conflict Resolution . . . 30
5.4 External APIs / SDKs . . . 30
5.5 User Interface Implementation . . . 31
5.5.1 UI Design Approach . . . 31
5.5.2 Screen-wise Implementation . . . 31
5.5.2.1 Screen 1: POS Checkout Screen . . . 31
5.5.2.2 Screen 2: Analytics Dashboard . . . 31
5.6 Deployment . . . 31
5.7 Chapter Summary . . . 31

6 Testing and Evaluation **33**
6.1 Testing Strategy . . . 33
6.2 Unit Testing . . . 33
6.3 Integration Testing . . . 33
6.4 System Testing . . . 34
6.5 Performance Testing . . . 34
6.6 Security Testing . . . 34
6.7 User Acceptance Testing . . . 35
6.8 Testing Summary . . . 35

7 Conclusion and Future Work **36**
7.1 Conclusion . . . 36
7.2 Future Work . . . 36
7.3 Limitations . . . 37

Appendices **39**
Appendix A – Turnitin Similarity Report . . . 39
Appendix B – AI Writing Detection Report . . . 41
Appendix C – Source Code . . . 43

List of Tables

1.1 Comparative Analysis of Existing System 2
1.2 Tools and Technologies Used in the Project 3

3.1 User Classes and Characteristics 10
3.2 UC-1.1: Intelligent Sales Processing 11
3.3 UC-1.2: Cloud Data Synchronization 12
3.4 Requirement Traceability Matrix 14

4.1 Data Dictionary – `products` Table. 27

5.1 External APIs Used 30
5.2 Deployment Details 31

6.1 Unit Test Case Summary 33
6.2 Integration Testing Summary 34
6.3 Performance Metrics 34
6.4 User Feedback Summary 35

List of Figures

3.1 Use Case Diagram of the Smart POS System 11

4.1 Context Diagram of the Smart POS System. 16
4.2 Layered Architecture Diagram (Clean Architecture). 17
4.3 Activity Diagram – Sales Processing Workflow. 18
4.4 UML Class Diagram for the Smart POS System 19
4.5 Sequence Diagram – Complete Transaction Flow. 20
4.6 Sequence Diagram – Google Sheets Synchronisation Flow. 21
4.7 State Transition Diagram – Product Lifecycle. 22
4.8 Level 0 DFD – Context and System Boundaries. 23
4.9 Level 1 DFD – Detailed System Operations. 24
4.10 Entity Relationship Diagram (ERD). 25
4.11 Physical Database Schema Diagram. 26

5.1 Visual Representation of the Smart POS Interface (Premium UX) 32

1 Turnitin Similarity Report Screenshot 40
2 Turnitin AI Writing Detection Report Screenshot 42

Chapter 1
Introduction

1.1 Introduction

Today, running a store means managing a lot of moving parts at once and doing it quickly. But a lot of small and medium-sized businesses (SMEs) still have to deal with old-fashioned manual bookkeeping, messy inventory spreadsheets, and checkout processes that annoy both employees and customers. There is enterprise-level ERP software, but it is usually expensive, bloated, and made for corporate IT departments instead of a store owner who just needs things to**work**.

Most cheap POS alternatives depend on the cloud, which means that if your WiFi connection is weak, your whole business could come to a halt. And even when they are online, many of these tools have interfaces that are so old that it takes a lot of work to train new staff to use them.

That's the gap the **Smart POS System** was designed to fill. Built with Flutter, it delivers a polished, high-performance experience on both mobile and desktop, while storing all critical data locally in SQLite so the business never skips a beat even offline. When an internet connection *is* available, it seamlessly syncs to Google Sheet for cloud backup and team visibility. Barcode scanning, PDF invoicing, and real time analytics round out a toolkit that put serious business software within reaches of any retailer.

1.2 Vision Statement

The main goal of this project is to break down the "Digital Barrier," which is the idea that only big companies with big budgets can use professional-grade business tools. Small business owners, restaurant managers, and warehouse managers should be able to get the same level of insight and automation as big businesses, without having to pay a monthly fee or have a consultant on speed dial.

The long-term goal is to create a self-sustaining ecosystem where a business can handle everything, from the first purchase order to the last receipt from a customer, completely offline. They can also send data to the cloud whenever they want.The Smart POS System wants to be a tool that people actually enjoy using by treating the interface as a product in itself (not an afterthought). It does this by providing clean data and real-time feedback that helps people make better decisions.

1.3 Related System Analysis

Before designing any solution, it is worth being honest about what already exists and where it falls short. Table 1.1 summarizes the most relevant competing approaches and the specific improvements this system targets.

What sets this system apart is its Clean Architecture foundation a modular design that makes the codebase easy to maintain and extend without breaking existing features. The business keeps

Table 1.1: Comparative Analysis of Existing System

System Type	Identified Weakness	How Smart POS Improve It
Standard Cloud POS	High per transaction fees limited or broken functionality without an active internet connection	One time local setup with SQLite for high speed offline operation no ongoing fees
Legacy Desktop POS	Outdated, non responsive UI; cumbersome to sync data across multiple devices.	Modern Premium Widget library; optional Google Sheets sync for multi device visibility.

running even when the internet does not while still having access to the analytics and reporting that modern retail demands.

1.4 Project Deliverables

The following five deliverables together form the complete Smart POS ecosystem:

1. **Cross-Platform Application Interface:** A high-fidelity mobile and desktop app with custom Premium Widgets covering inventory management, sales processing, and live analytics.
2. **Automated Reporting Engine:** A module that generates dynamic PDF invoices on demand and exports comprehensive sales histories to Excel/CSV for further analysis.
3. **Cloud Sync Module:** Google Sheets API integration that backs up local sales data to a free cloud environment, accessible from any browser.
4. **Scanning & Printing Suite:** Built in barcode/QR scanner support and thermal printer integration (Bluetooth and WiFi) for physical receipt generation at the counter.
5. **Technical Documentation:** A full documentation package including the Software Requirements Specification (SRS), Software Design Document (SDD), and User Guide

1.5 System Limitations and Constraints

Every system has its boundaries, and it is important to be upfront about them:

- **Hardware dependency:** Physical receipt printing requires a compatible Bluetooth or WiFi thermal printer the app itself does not provide one.
- **Initial authentication:** The first-time Google Sign-in and the initial cloud sync both require an active internet connection after that, offline mode takes over
- **Storage capacity:** The local SQLite database is limited only by the available internal storage on the host device (smartphone or PC), which varies widely.
- **Platform-specific features:** Biometric login relies on the device having the appropriate sensor hardware (fingerprint reader, Face ID, etc.).

1.6 Tools and Technologies

Every technology in the stack was chosen for a clear reason performance, developer productivity, or cost. Table 1.2 documents those choices.

Table 1.2: Tools and Technologies Used in the Project

Tool / Technology	**Version**	**Purpose**
Flutter SDK	3.x	Cross-platform reactive UI framework one codebase for Android, iOS, Windows, and macOS.
BLOC / Provider	8.1.3	Predictable state management that keeps business logic cleanly separated from the UI layer.
SQLite (sqflite)	2.3.0	Lightweight relational database for fast, structured, offline first local storage.
Google Sheets API	v4	Free cloud layer for data backup, sharing, and multi device reporting without a dedicated server.

1.7 Relevance to Course Modules

The Smart POS System is not an isolated exercise its a practical proof of concept that brings together concepts from across the computer science curriculum.

1.7.1 Programming Fundamentals

Object Oriented Programming (OOP) runs throughout the codebase. Every key business entity Product, Invoice, Staff member is modelled as a class with well defined properties and behaviors. Separating business logic from the UI keeps the code readable today and maintainable a year from now.

1.7.2 Data Structures and Algorithms

Efficiency matters in a POS context. Custom algorithms handle real time tax calculations and running totals, while optimized search and sort routines let users filter a catalog of thousands of products by name, category or barcode in a fraction of a second.

1.7.3 Database Management Systems

The SQLite schema adheres to solid relational design principles: primary keys, foreign keys, and one-to-many relationships connect Sales to Invoices to individual line-item Products. Normalization maintains data consistency and ensures queries run efficiently, even as the database scales.

1.7.4 Software Engineering

The project followed an Agile based SDLC from the start. Requirements were captured in a formal SRS document, and the architecture was mapped out with UML diagrams Use Case, Sequence, and Class before a line of implementation code was written. This upfront investment paid off in fewer surprises during development

1.7.5 Human Computer Interaction (HCI)

It was a conscious choice to add the Premium Widgets library and Lottie animation integration, with HCI in mind.The goal was to make the point of sale easier to use by making every tap feel responsive and clear. This would also cut down on the time it takes for a new employee to get used to the interface.

1.8 Chapter Summary

This chapter prepared the way for everything that came after it. It showed why a local-first Smart POS system is worth making by showing the real-world problems it solves and the holes it fills in the current market. The vision was made clear, the most important tasks were listed, and the technical and academic bases were laid out clearly.

Chapter 2 then goes into more detail about the problem: it breaks down exactly what the system needs to do, who it needs to serve, and the functional requirements that will guide all design and implementation decisions that come after.

Chapter 2
Problem Definition

2.1 Problem Statement

This project is mainly about the problem of small and medium-sized businesses (SMEs) not having access to affordable, reliable, and easy-to-use digital transaction tools. Many small businesses still use paper-based ledgers or manual cash registers, which can make mistakes, lose data, and don't show sales trends or inventory levels. Shop owners who have trouble keeping track of their money and employees who have to deal with long lines at the checkout during busy times are two groups that are affected.

The main goal of this project is to help small and medium sized businesses (SMEs) get digital payment tools that are inexpensive, dependable, and simple to use. A lot of small businesses still use paper based ledgers or manualcash registers. These can make mistakes, lose data, and not shows sales trends or inventory levels. Two groups that are affected are shop owners who have trouble keeping track of their money and employees who have to deal with long lines at the checkout during busy times. Many cloudbased POS systems don't work well in places with spotty internet connections, which can lead to downtime and unhappy customers. There is a big difference between the basic tools that small businesses can use and the high-end software that big businesses can use. This means that there needs to be a professionalgrade, local-first system that keeps data safe and keeps the business running even when the network is down.

2.2 Proposed Solution Overview

The proposed Smart POS System is a cross platform application built using the Flutter framework, providing a high performance "local-first" architecture. By utilizing an embedded SQLite database, the system ensures that sales processing and inventory management can occur entirely offline with native-speed performance. To address the need for data backups and remote reporting, the system integrates the Google Sheets API as a free and accessible cloud synchronization layer. The innovation lies in its "Premium Design System"—a library of custom widgets that offers an intuitive, engaging interface usually reserved for expensive SaaS products. This solution improves efficiency by automating the checkout process, reducing manual errors through barcode scanning, and providing instant analytical insights into business health through dynamic charts and automated reports.

2.3 Objectives of the Proposed System

The following objectives have been established to ensure the system effectively addresses the identified retail challenges:

Business Objectives (BO):

- BO-1: Develop a high performance cross platform application for Windows and Android using Flutter.
- BO-2: Improve transaction speed by implementing an intuitive POS interface with barcode scanning support.
- BO-3: Provide real time inventory tracking to prevent stock outs and overstocking through automated alerts.
- BO-4: Ensure 100% offline operational reliability by using a robust local SQLite relational database.
- BO-5: Set up your system to automatically make digital invoices in PDF format and sales summaries in Excel or CSV format.
- BO-6: Synchronize business data with Google Sheets to provide a secure, cost-free cloud backup solution.

2.4 Scope of the System

The Smart POS System is a full point of sale and inventory management tool that is made just for small stores, cafe and specialty shop. The system will work with Windows (Desktop), Android (Tablet/Mobile), and the Web. Some of the most important features are managing the product catalog, keeping track of suppliers, secure multi-role authentication and a changing checkout system. The system will work with outside hardware like thermal printers and barcode scanners. The system, on the other hand, does not have modules for managing payroll or complicated shipping and freight logistics. You can only work within the operational limits of a single store or multi counter retail setup where speed and ease of use of local data are the most important things.

2.5 System Architecture and Modules

The Smart POS System follows a **Clean Architecture** pattern, emphasized modularity and the separation of concerns. The system is divided into three primary layers: the **Data Layer** (handling SQLite and API interactions), the **Domain Layer** (containing core business logic and entities) and the **Presentation Layer** (managing the UI states via BLoC/Provider). This modular design allows for independent testing and maintenance of features like inventory, sale and analytics. Data flow is undirectional, ensuring that UI updates are predictable and reactive to the underlying database state.

2.5.1 Module 1: Authentication & Security

This module manages the system entry point and internal security. It supports multi-layered authentication including Google Sign-in for cloud access and local biometric/PIN authentication for quick store floor access.

Functional Requirements:

- FR-1: The system shall allow administrators to register staff and assign specific roles.
- FR-2: The system should authenticate user before granting access to sensitive financial data.
- FR-3: The system should keep an audit log of all login and logout activities.

2.5.2 Module 2: Inventory & Product Management

This module acts as the digital catalog for the business, allowing for the comprehensive management of products, stock levels, and suppliers relationship.

Functional Requirements:

- FR-1: The system should enable CRUD (Create, Read, Update, Delete) function on product
- FR-2: The system should support image uploads and barcode assignment for each product.
- FR-3: The system shall track stock levels and trigger alert when items fall below a defined thres hold.

2.5.3 Module 3: Sales Engine (Core Module)

This is the primary operational module, providing the POS interface where transactions are processed. It handles cart management, discount applications, and final bill calculation.

Functional Requirements:

- FR-1: The system should process transactions by scanning barcode or selecting item manually.
- FR-2: The system should calculate total amount, taxe, and change due in real time.
- FR-3: The system shall generate a digital PDF receipt upon successful completion of a sale.

2.5.4 Module 4: Analytics & Reporting (Admin Module)

This module provide administrator with strategic insights into business performance through visual dashboard and exportable data file.

Functional Requirements:

- FR-1: The system shall display real time sales chart and profit margin on the dashboard.
- FR-2: The system shall generate daily, weekly, and monthly sale report in PDF and Excel format .
- FR-3: The system should provide a filtered breakdown of sales by product category or staff members.

2.5.5 Module 5: Integration & Cloud Service

This module handle communication with external entitie, primarily focusing on hardware interaction and cloud synchronization.

Functional Requirements:

- FR-1: The system shall synchronize local SQLites data with Google Sheet for backup purpose.
- FR-2: The system should interact with external thermal printers for physical receipt printings
- FR-3: The system should detect and notify user of connectivity stat for cloud feature

2.6 Assumptions and Dependencies

The design and development of the system rely on the following assumptions and external dependencies:

- The system assumes the host device has sufficient internal storage to maintain the SQLite database.
- Cloud synchronization depends on the availability and API rate limits of the Google Sheets services.
- Physical printing assumes the use of standard ESC/POS compatible thermal printer via Bluetooth or USB.
- The system assumes that administrator have a Google account for initial setup and cloud reporting

2.7 Chapter Summary

This chapter defined the retail operational inefficiencies addressed by the Smart POS System, including manual errors and cloud dependency. A local first solution using SQLite and Google Sheets was proposed. Clear business and technical objectives were established, and the system modular architecture was outlined. Assumption about hardware and connectivity were documented

This chapter provides the problem foundation and sets the direction for the detail software design presented in the next chapter

Chapter 3
Requirement Analysis

3.1 Introduction to Requirement Analysis

Requirement Analysis serves as the critical bridge between the high level business vision and the concrete technical implementation of the Smart POS System. This chapter provides a rigorous specification of the system functional behavior, quality attribute and interface constraint. Given the mission-critical nature of Point of Sale software where data loss or UI latency directly affects business revenue this analysis prioritizes robustness, high speed interaction and reliable data persistence.

By documenting the specific needs of retail stakeholders, ranging from inventory accuracy to secure financial reporting this chapter establishes measurable evaluation criteria for the entire system. Each requirement is assigned a unique identifier to support traceability throughout the Software Development Life Cycle ensuring that no project objective goes unverified during implementation or testing.

3.2 Requirement Elicitation Techniques

To capture the complex and often implicit needs of a modern retail environment, a multi faceted elicitation strategy was employed. This approach ensured that requirements reflected both technical benchmark and real world operational workflow.

- **Structured Stakeholder Interviews:** Retail managers were interviewed with a particular focus on the "Speed to Checkout" metric. Findings revealed that manual barcodes entry is a significant operational bottleneck, directly motivating the requirement for native speed HID scanner integrations.
- **Operational Workflow Observation:** Direct observation of cashier routines in local grocery stores identified the need for a dedicated "Low-Connectivity Mode." This insight led to the decision to implement local-first storage with background synchronization, reducing dependency on realtime API connectivity.
- **Domain Document Analysis:** Existing paperbased invoices, tax records, and inventory ledgers were reviewed and analyzed. This informed the design of a normalized SQLite database schema that accurately reflects local fiscal and accounting requirements.
- **Competitive Benchmarking:** A review of leading POS platforms such as Square and Loyverse highlighted the commercial importance of rich visual feedback including animations and haptic responses. These findings informed the development of the project specialized "Premium Widgets" library.
- **Technical Feasibility Study:** The capabilities and limitations of the Flutter framework and Google Sheets API were evaluated to establish realistic constraints on data synchronization frequency and local database capacity.

3.3 User Classes and Characteristics

The system is designed to serve range of user each with distinct interaction pattern and data access privilege. Table 3.1 summarize these role and their technical profiles.

Table 3.1: User Classes and Characteristics

User Class	Description	Technical Level
Business Owner	Holds full administrative rights. Responsible for high level financial oversight, staff payroll configuration, and cloud backup management.	Intermediate / Advanced
Store Cashier	The primary operator of the Sale Module. Handles high speed transaction processing, receipt generation, and customer registration.	Basic / Novice
Stock Controller	Operates within the Products and Suppliers modules. Responsible for stock entry, supplier audit and scheduling of promotional offers.	Intermediate
System Auditor	A non-operational, read only role with access to audit logs and detailed transaction history, supporting financial transparency and .	Technical / Advanced

3.4 Use Case Model

The use case model defines the interaction boundaries between human actors and the system core module, with a particular focus on three primary retail domains: Sale, Inventory, and Analytic.

3.4.1 Use Case Diagram Description

The use case diagram identifies two primary actors—Administrator and Staff interacting within the system boundary. The "Process Sale" use case incorporates the "Verify Stock" use case through an *include* relationships, while access to "View Analytic" is conditionally governed by an *extend* relationship tied to user authorizations

3.4.2 Detailed Use Case Specifications

To facilitate developer clarity and reduce ambiguity during implementation, the major use cases are broken down into their logical step-by-step flows below.

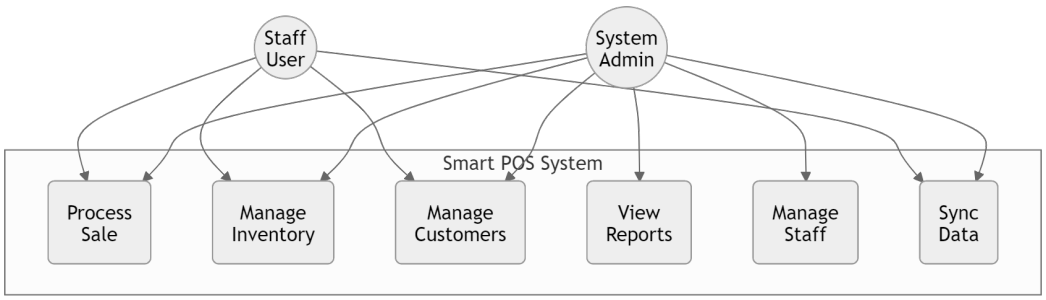


Figure 3.1: Use Case Diagram of the Smart POS System

Table 3.2: UC-1.1: Intelligent Sales Processing

Use Case ID	UC-1.1
Actors	Cashier, Administrator
Preconditions	1. The user is authenticated. 2. The local SQLite database has been successfully initialized.
Normal Flow	1. The cashier scans a product barcode using an external HID scanner or the device's built-in camera. 2. The system queries the SQLite database for a matching SKU or GTIN. 3. The matched item is added to the reactive cart and the running subtotal is updated instantly. 4. The cashier applies any active promotions or discounts retrieved from the database. 5. The cashier selects the payment method: Cash, Card, or Digital Wallet. 6. The system finalizes the invoice, decrements stock levels, and prepares the digital receipt in PDF format.
Postconditions	1. The database is updated with the completed order record. 2. A digital receipt is stored in local storage.

3.4.2.1 UC-1.1: Intelligent Sales Processing

3.4.2.2 UC-1.2: Cloud Synchronization and Backup

3.5 Functional Requirements

3.5.1 Module 1: Authentication and Audit Systems

- **FR-1.1:** The system shall support Google-based Sign-In for administrative accounts to enable seamless cloud service linking.
- **FR-1.2:** The system shall provide rapid local Biometric and PIN-based authentication, completing store access in under five seconds.
- **FR-1.3:** The system shall maintain a tamper-evident audit log for every stock modification, capturing the User ID, timestamp, and change delta.

Table 3.3: UC-1.2: Cloud Data Synchronization

Use Case ID	UC-1.2
Actors	Business Owner, Background Service
Description	Local transaction data is securely mirrored to a designated Google Sheet cloud instance to ensure business continuity and remote accessibilitys.
Normal Flow	1. The background connectivity service detects an active internet connection. 2. The system identifies "Dirty" rows un synced sales records within the local database. 3. The system authenticates with Google Cloud using stored OAuth2 token 4. Transaction data is pushed to the designated sheet via JSON payloads. 5. Upon successful upload, the corresponding local records are marked as "Synced."
Postconditions	Local database records are marked as synced, and the Google Sheet is updated with the new transaction rows.

3.5.2 Module 2: Inventory and Multi-Supplier Management

- **FR-2.1:** The system shall support the creation of hierarchical product categories and subcategories to enable efficient filtered search.
- **FR-2.2:** The system shall trigger automated UI notification when inventory level fall below a user defined minimum stock threshold
- **FR-2.3:** The system shall maintain supplier profiles directly linked to associated products to streamline the procurement workflow.

3.5.3 Module 3: Sales Engine and Promotions

- **FR-3.1:** The system shall support Active Promotions of both percentage-based and fixed-amount types, with automatic application to the shopping cart.
- **FR-3.2:** The system shall generate a unique UUID for every invoice to eliminate the possibility of transaction ID collision
- **FR-3.3:** The system shall support a "Parked Sale " feature, enabling a cashier to temporarily hold an active cart and processes a separate transaction without data loss

3.6 Non-Functional Requirements

3.6.1 Performance Requirements

- **PER-1: Search Latency.** A product database query across a catalog of 5,000 item shall return result with in 150 millisecond

- **PER-2: Frame Rates** The application UI shall sustain a consistent 60 frames per second during all animations and screen transitions, preserving the intended premium user experience.
- **PER-3: Cold Start.** The application shall reach a fully interactive state from a cold launch in under 2.5 seconds.

3.6.2 Security and Data Integrity

- **SEC-1: ACID Compliance.** All financial database transactions, including sales and invoices, shall adhere strictly to Atomicity and Durability principles to prevent data corruption in the event of unexpected power loss.
- **SEC-2: Role-Based Access Control (RBAC).** Access to the Profit/Loss Analytics module shall be restricted exclusively to users holding the "Owner" role, enforced through secure application middleware.

3.6.3 Availability and Connectivity

- **REL-1: Offline Autonomy.** The system shall remain fully operational for a minimum of 100 consecutive sales transactions without any external network dependency.
- **REL-2: Background Sync Resilience.** The background synchronization service shall automatically retry any failed cloud upload attempts within five minutes of network restoration.

3.7 External Interface Requirements

3.7.1 Users Interface Requirement

The UI shall be built upon the project's dedicated **Premium Widget Library**, designed to deliver a consistent and polished interaction model across all modules.

- **UI-1: Visual Feedback.** All interactive elements shall provide haptic feedback and micro-animations, implemented via the `flutter_animate` package.
- **UI-2: Adaptive Layout.** The interface shall automatically transition to a "Tablet Mode" layout—featuring a sid -by side cart and product catalog view on displays wider than 600dp.

3.7.2 Hardware Interface Requirements

- **HI-1: Barcode Scanning.** The system should integrate with device camera sensor to support QR code and barcode decoding via the `mobile scanner` packages.
- **HI-2: Receipt Printing.** The system shall support direct communication with 80 mm Bluetooth thermal printer for the generation of physical invoice copie.

3.8 Requirement Traceability Matrix

The Requirement Traceability Matrix (RTM) in Table 3.4 map each project objective to its corresponding use case and functional requirement ensuring fully coverage and verifiability throughout the development life cycle.

Table 3.4: Requirement Traceability Matrix

Project Objective	**Use Case**	**Associated Requirements**
Business Automation	UC-1.1	FR-3.1, FR-3.2, FR-2.1
Data Reliability	UC-1.2	FR-1.3, REL-1, REL-2
User Experience	UC-HCI	FR-PremiumWidgets, PER-2
Cloud Synchronization	UC-1.2	FR-CloudSync, SEC-2
Stock Governance	UC-Inventory	FR-2.2, FR-2.3

Chapter Summary

This chapter presented a comprehensive specification of the Smart POS System's requirements, with particular emphasis on its premium user experience and offline-first architecture. Four primary user roles were defined and their system interactions were captured through detailed use cases covering Intelligent Sales Processing and Cloud Synchronization. Functional requirements were organized into clearly scoped system modules, while non functional requirement established concrete benchmarks for performance, security, and data integrity. Hardware and software interface requirements further clarified the system operational boundaries. Together these specifications form a solid and traceable foundation for the technical design work presented in Chapter 4

Chapter 4
Software Design

4.1 Introduction to Software Design

This chapter presents the detailed software design of the Smart POS System. It translates the requirements identified in Chapter 3 into structured architectural, behavioural, and data models. The purpose of this chapter is to provide a clear blueprint for system implementation, ensuring maintainability, scalability, and modular development by utilising a *Clean Architectures* approach within the Flutter frameworks, the system is designed to be decoupled, testable and highly responsive to user interaction

4.2 Design Methodology and Software Process Model

4.2.1 Design Methodologys

This project follows the **Clean Architecture** design methodology. This approach was selected to ensure a strict separation of concerns between the business rules, data access, and the user interface.

- **Presentation Layer:** Handles all UI components and state management (BLoC).
- **Domain Layer:** Contains the core business logic (Use Cases) and entities. This layer is independent of any external libraries.
- **Data Layer:** Manages data persistence via SQLite and cloud synchronisation via the Google Sheets API.

The selected methodology ensures that the core business logic of the POS (e.g. tax calculations and inventory rules) remains unchanged even if the database backend or UI framework is replaced. **SOLID** principle are strictly followed, particularly the Single Responsibility Principles, to keep the codebases modular

4.2.2 Software Processes Model

The project follows the **Agile Scrum** model. Agile was chosen due to the iterative nature of POS system requirements. Development was broken down into Sprints, each focusing on a specific module (e.g. Sprint 1: Inventory, Sprint 2: Sales Engine). This allowed for continuous testing and feedback, ensuring that the barcode-scanning and receipt-generation features were refined through multiple iterations before final integration.

4.3 System Overview

The Smart POS System acts as a central hub for retail operations, interacting with local hardware (scanners/printers) and cloud services. Figure 4.1 illustrates the high-level context of the system.

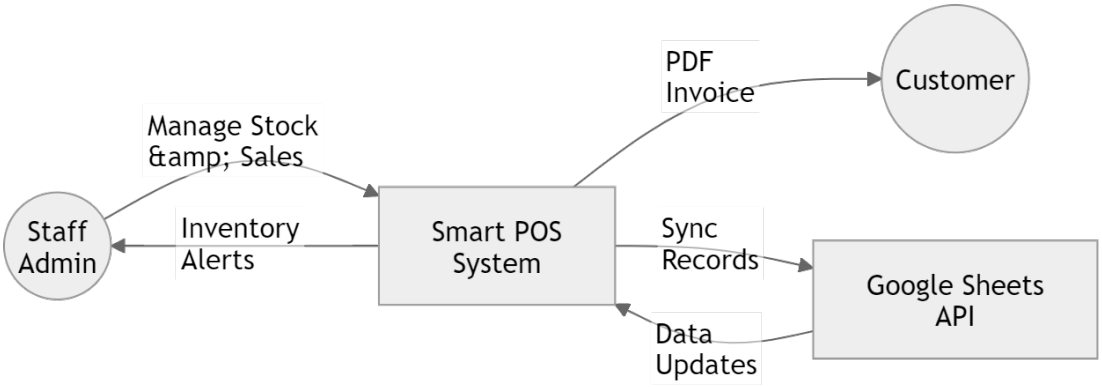


Figure 4.1: Context Diagram of the Smart POS System.

The system interacts with three primary actors:

- **Admin / Owner:** Manages inventory, views analytics, and configures cloud sync.
- **Cashier:** Processes daily sales and generates receipts.
- **External Systems:** Google Sheet (cloud backup) and thermal printer (output).

4.4 Architectural Design

The system architecture is designed as a *local first, cloud synced* system. The layered Clean Architecture diagram is shown in Figure 4.2.

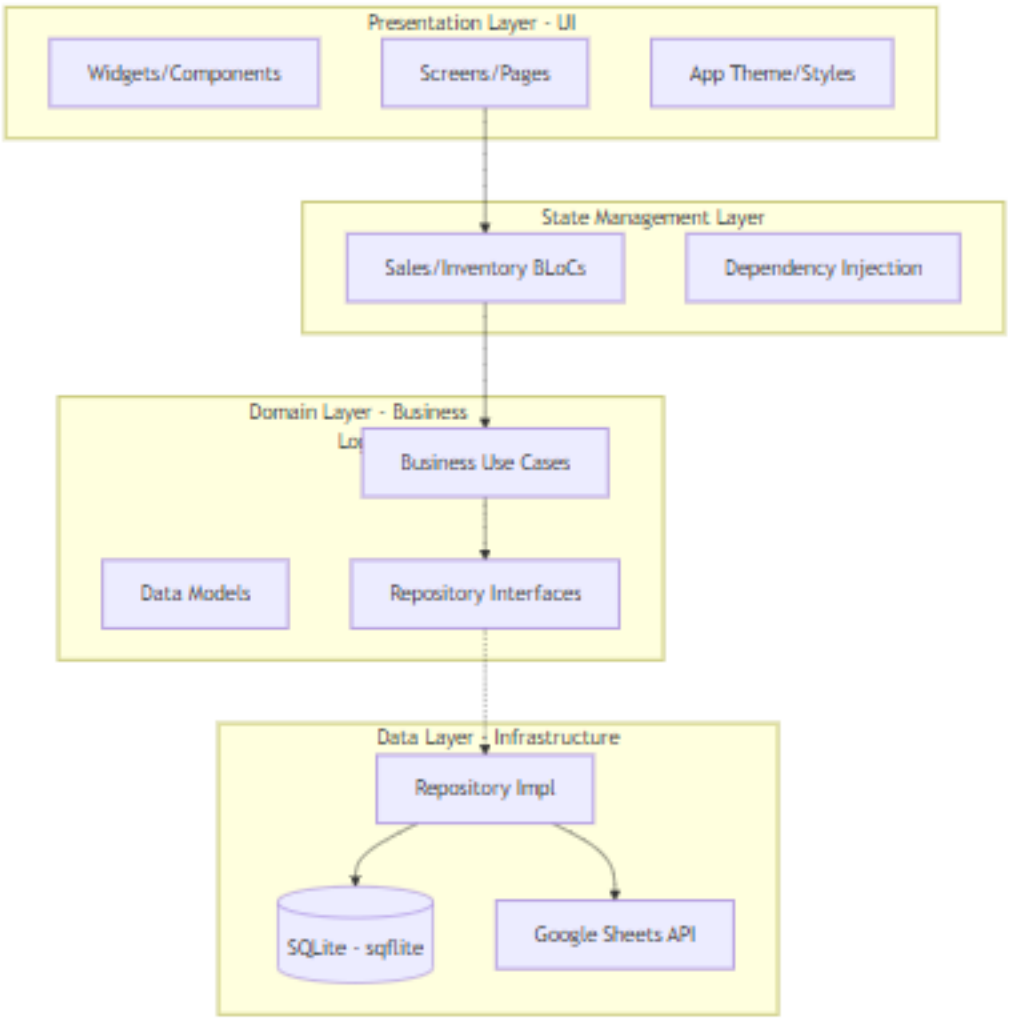


Figure 4.2: Layered Architecture Diagram (Clean Architecture).

The architecture consists of three primary horizontal layers:

- **Frontend (UI Layer):** Built with Flutter widgets, utilising the BLoC patterns for reactive state update.
- **Business Logic Layer:** Contains use cases for *Process Sale*, *Update Stock*, and *Generate Report*.
- **Persistence Layer:** Utilises SQLite for mission critical local storage and a sync service for Google Sheet integrations.

4.5 Design Models

This section presents the behavioural and structural models of the system using standard UML notation.

4.5.1 Activity Diagrams

Activity diagrams represent the workflow of the two most critical modules: Sales and Inventory.

4.5.1.1 Module 1: Sales Engine Workflow

Figure 4.3 shows the complete sales-processing workflow.

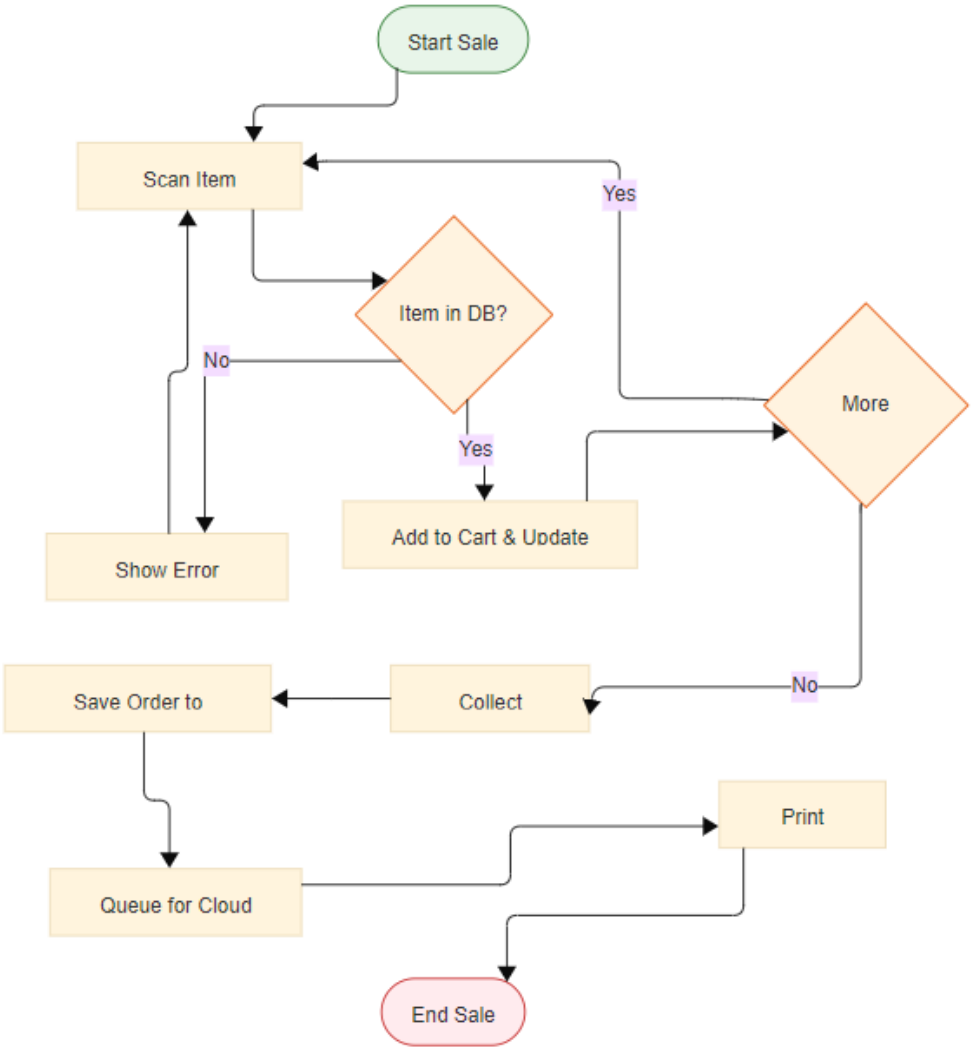


Figure 4.3: Activity Diagram – Sales Processing Workflow.

The workflow proceeds as follows:

1. Cashier scans a barcode
2. System check whether the item exists in local SQLite
3. If found the item is added to the reactive cart otherwise a *Not Found* error is display
4. The total is calculated in real time
5. On *Complete Sale*, the system updates stock levels and trigger the printer

4.5.2 Class Diagram

The class diagram (Figure 4.4) shows the static structure of the domain entities and their relationships

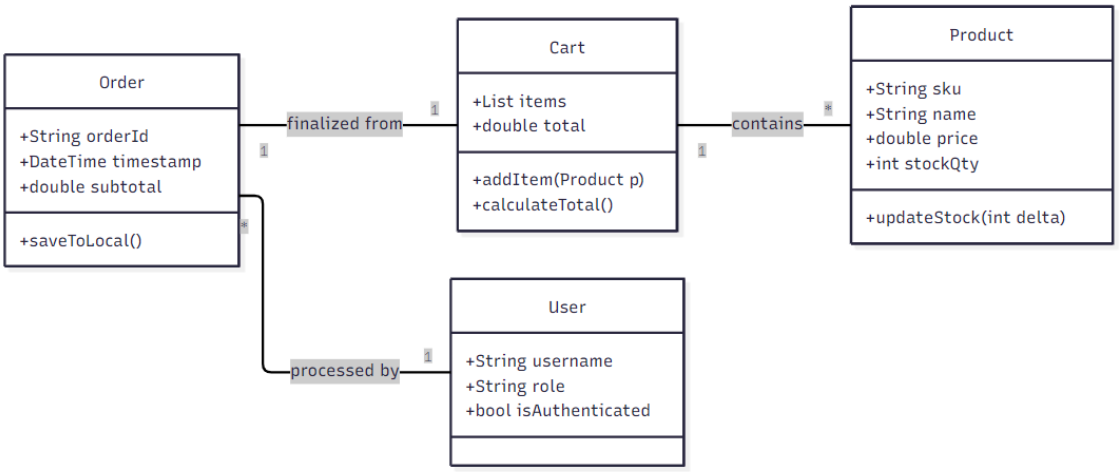


Figure 4.4: UML Class Diagram for the Smart POS System

Key classes include:

- **Product:** Represents a retail item with SKU, name, price, and stock quantity
- **Cart:** A container for `CartItem` objects with methods `addItem()` and `calculateTotal()`.
- **Order:** Represents a completed transaction linked to multiple products.
- **DatabaseHelper:** A singleton class managing the SQLite connection

4.5.3 Sequence Diagrams

Sequence diagrams illustrate the time-sequenced interaction between objects during a specific use case.

4.5.3.1 Sequence Diagram – Processing a Sale

Figure 4.5 depicts the complete transaction flow. The UI sends an `AddProduct` event to the `SalesBloc`, which calls the `InventoryRepository` to fetch data. Once the sale is finalised, the `OrderRepository` saves the record and notifies the `SyncService`.

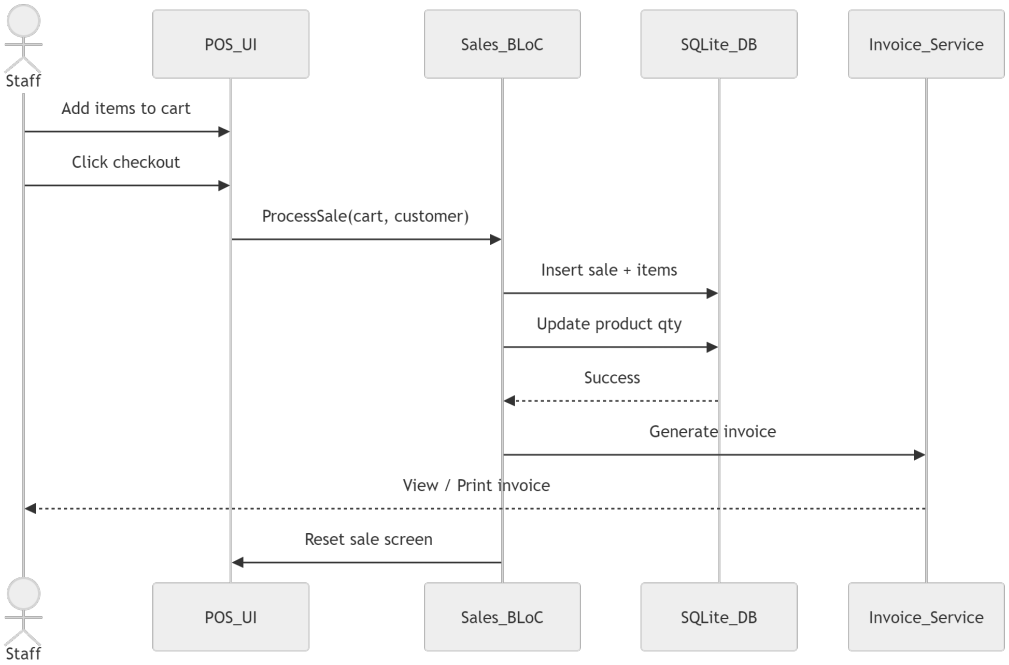


Figure 4.5: Sequence Diagram – Complete Transaction Flow.

4.5.3.2 Sequence Diagram – Google Sheets Synchronisation

Figure 4.6 illustrates the background process of synchronising local sales data with Google Sheet, ensuring cloud redundancy.

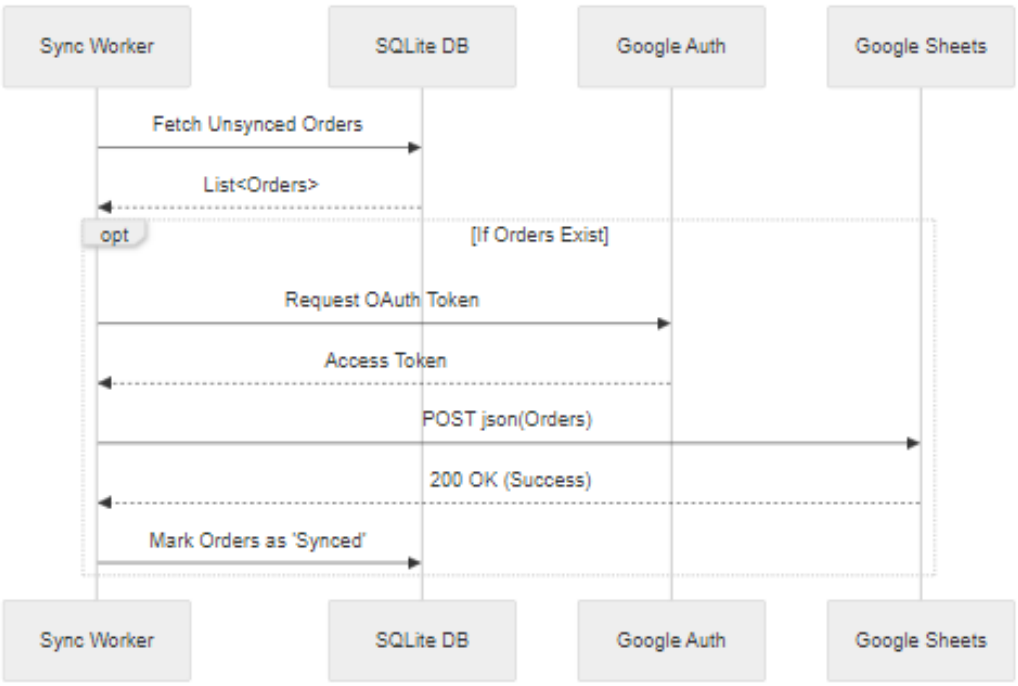


Figure 4.6: Sequence Diagram – Google Sheets Synchronisation Flow.

4.5.4 State Transition Diagram

Figure 4.7 tracks the lifecycle of a Product entity through its possible states.

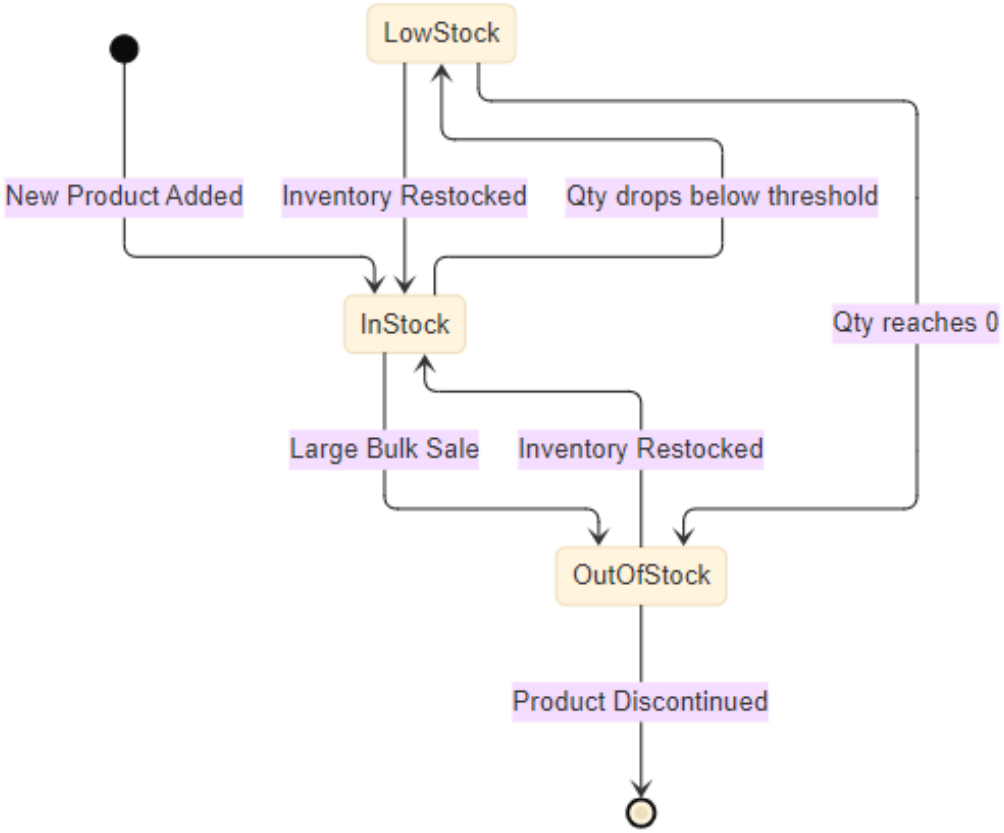


Figure 4.7: State Transition Diagram – Product Lifecycle.

The diagram models three states: `InStock` → `LowStock` (triggering a warning) → `OutOfStock` (blocking further sales).

4.6 Data Flow Diagrams

4.6.1 Level 0 Data Flow Diagram (Context DFD)

Figure 4.8 presents the highest-level view of the system, showing its boundaries and external entities.

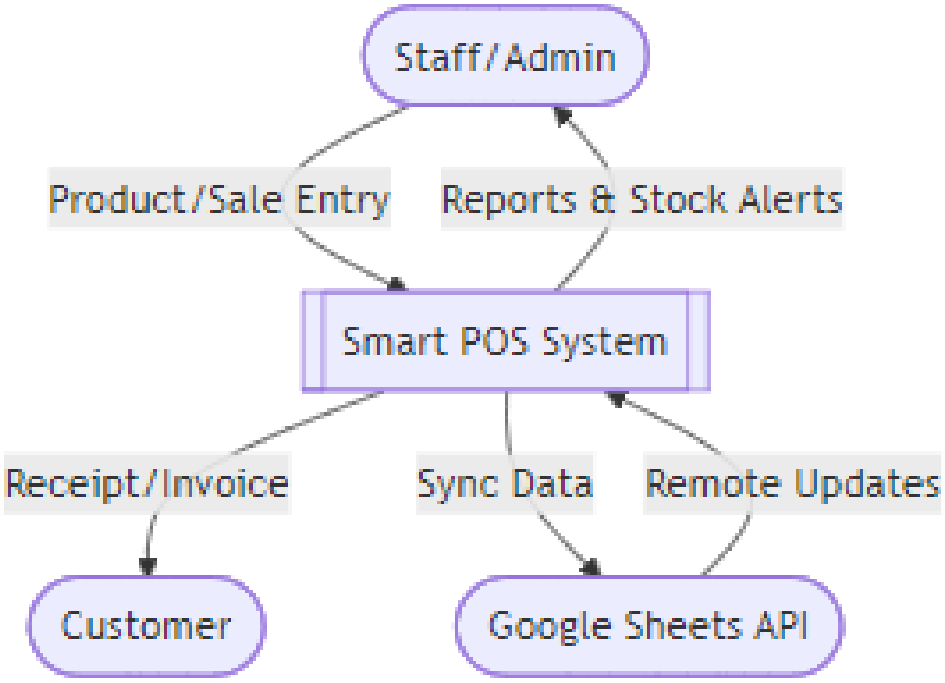


Figure 4.8: Level 0 DFD – Context and System Boundaries.

4.6.2 Level 1 Data Flow Diagram

The Level 1 DFD (Figure 4.9) provides a granular view of the system processes, detailing how inventory data and sales events flow through the system.

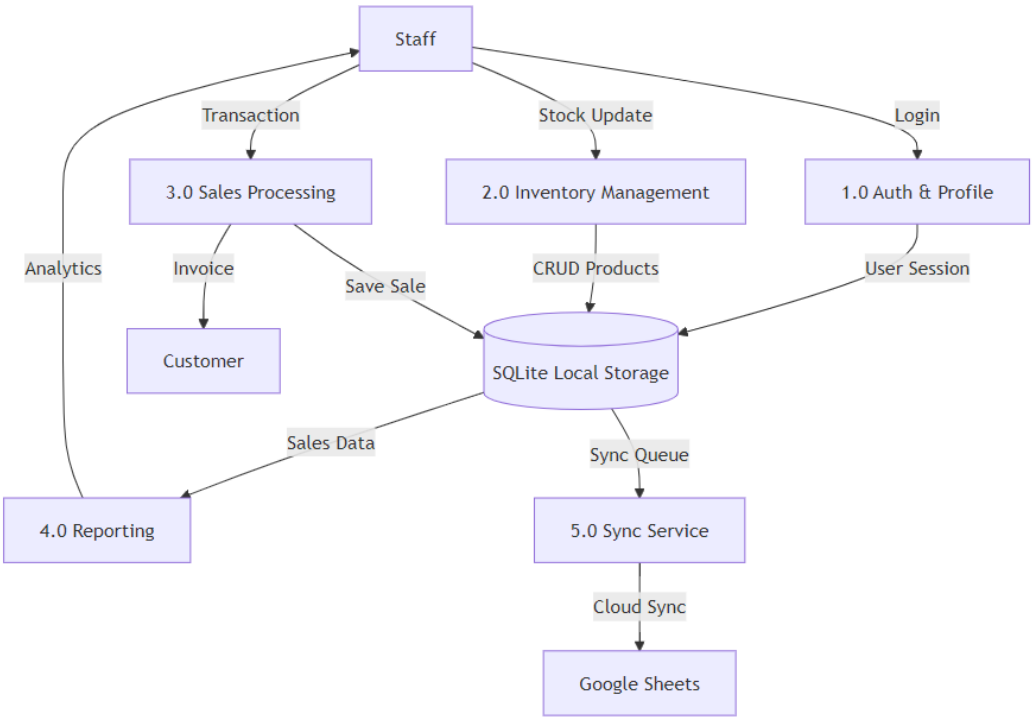


Figure 4.9: Level 1 DFD – Detailed System Operations.

4.7 Data Design

The Smart POS System uses a **relational database (SQLite)** for local storage to ensure ACID compliance during financial transactions.

4.7.1 Entity Relationship Diagram (ERD)

Figure 4.10 shows the structural relationships between the main data entities.

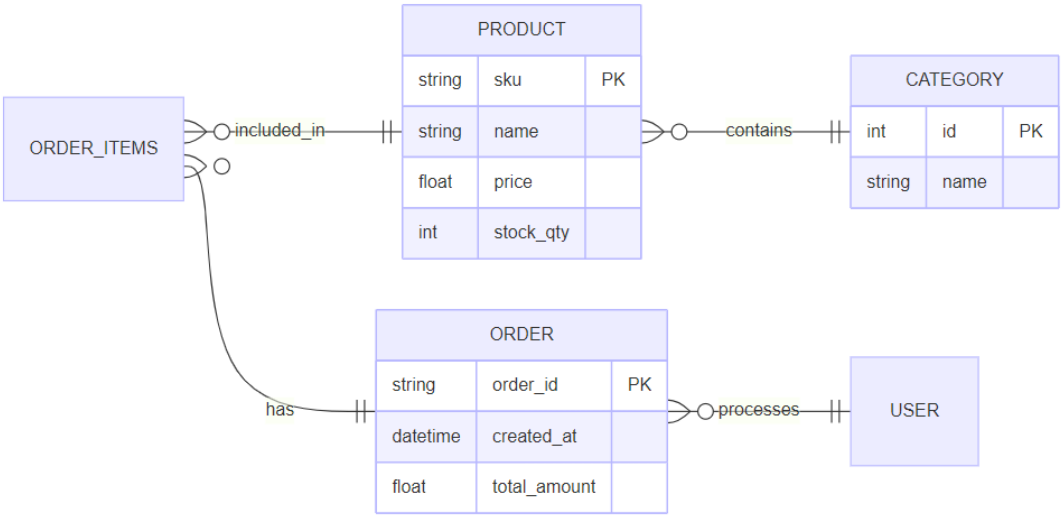


Figure 4.10: Entity Relationship Diagram (ERD).

The ERD shows a **One-to-Many** relationship between `Category` and `Product`, and a **Many-to-Many** relationship between `Product` and `Order` (resolved via an `OrderItems` junction table).

4.8 Database Schema Diagram

Figure 4.11 presents the physical database schema, reflecting the tables and foreign-key constraints implemented in SQLite.

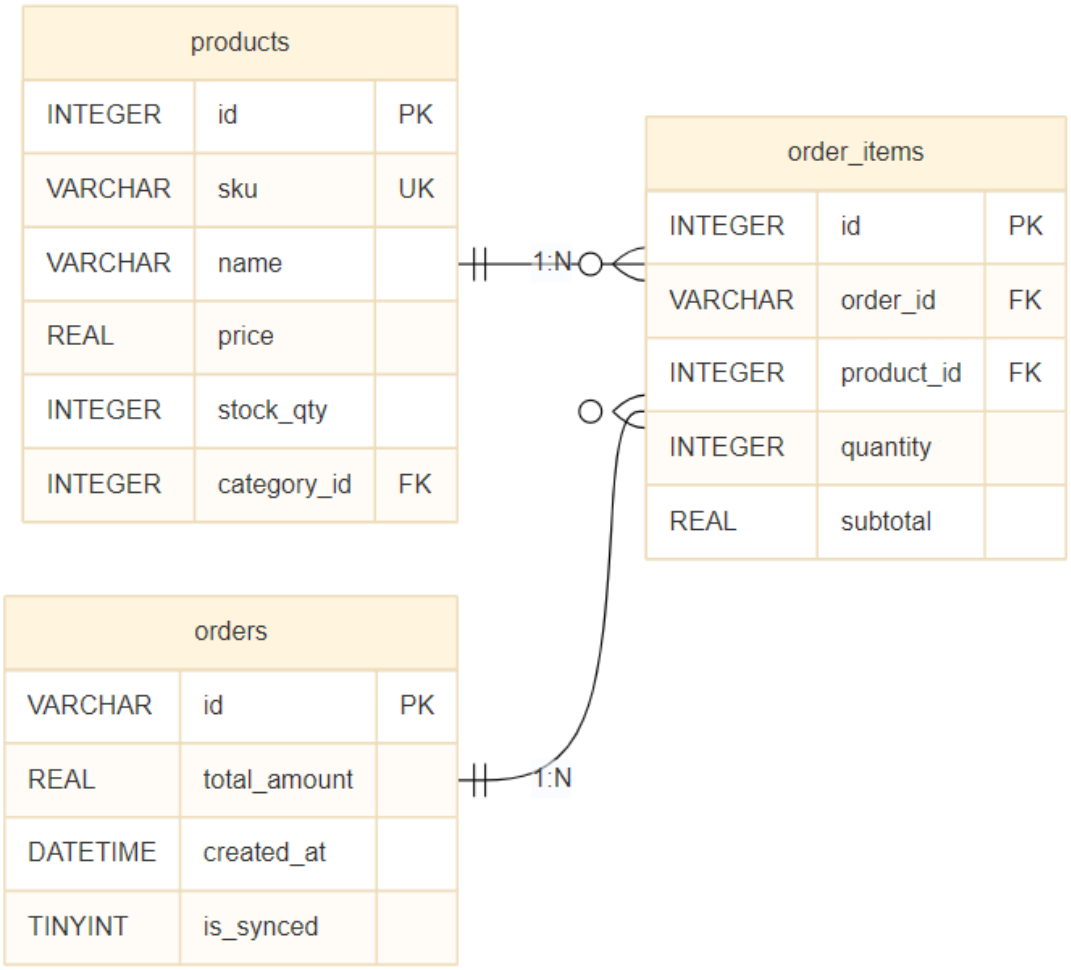


Figure 4.11: Physical Database Schema Diagram.

The schema ensures data integrity through foreign-key constraints on the `product_id` column in the `order_items` table. All monetary values are stored as `INTEGER` (cents) to avoid floating-point precision errors.

4.8.1 Data Dictionary

Table 4.1 describes the fields of the `products` table.

Table 4.1: Data Dictionary – `products` Table.

Field Name	**Data Type**	**Size / Key**	**Description**
`id`	INTEGER	PK (AUTO)	Unique auto-incrementing identifier.
`sku`	VARCHAR	50	Barcode or serial number of the product.
`name`	VARCHAR	100	Human-readable product name.
`price`	REAL	–	Unit selling price (stored in cents as INTEGER in practice).
`stock_qty`	INTEGER	–	Remaining quantity available in stock.

4.9 Chapter Summary

This chapter laid out a thorough design framework for the Smart POS System. By adopting a Clean Architecture foundation alongside an Agile Scrum methodology the system is built to be both resilient and flexible. A suite of UML diagrams including Activity, Class, Sequence and State Transition model was used to capture the application structural and behavioral dimensions in depth The data design introduced a relational SQLite schema engineered to maintain financial precision while supporting offline functionality. Together, these design decisions form the groundwork that drives the implementation work covered in Chapter 5

Chapter 5
Implementation

This chapter describes the practical implementation of the Smart POS System. It explains how the Clean Architecture design was transformed into a fully functional Flutter based application, detailing the development environment, core modules, algorithms, UI components and final deployment strategy

5.1 Development Environment

The Smart POS System was developed using modern cross-platform technologies to ensure smooth performance on both mobile and desktop platforms.

- **IDE:** Visual Studio Code, Android Studio
- **Programming Language:** Dart 3.x
- **Framework:** Flutter SDK
- **State Management:** BLoC and Provider
- **Database:** SQLite using the `sqflite` plugin
- **Operating System:** Windows 11 (Development), Android (Deployment)
- **Hardware:** Intel i7 Processor, 16GB RAM, Android Tablet for testing

5.2 Core Module Implementations

The system was designed as a collection of loosely coupled modules so that updates to UI layers do not affect underlying business rules.

5.2.1 Module 1: Reactive Sales Engine

The Sales Engine form the core of the POS workflow. It uses the BLoC pattern to manage the realtime shopping cart. Upon scanning a barcode, an event is dispatched to the `SalesBloc`, which retrieves the corresponding product from SQLite and updates the reactive data stream. The UI listens to this stream and automatically rebuilds totals and stock indicator

5.2.2 Module 2: Structured Inventory Management

This module handles CRUD operations for products and suppliers.It include advanced form validations to ensure:

- SKU values remain unique

- Pricings format follow currency rule

An internal "Image Services" manages local file selection and Base64 encoding for product icons.

5.3 Algorithm Implementation

The POS system uses optimized algorithms to maintain accuracy and efficiency during high-volume transactions.

5.3.1 Algorithm 1: Real time Cart Aggregation

This algorithm calculates the subtotal, tax, and final grand total for multi item carts.

Input: List of `CartItem` objects containing `unitPrice`, `quantity`, and `taxRate`. **Output:** Calculated subtotal, total tax, and grand total.

Pseudo Code:

```
Algorithm CalculateOrderTotal(CartItems):
1. Initialize subtotal = 0, totalTax = 0
2. For each item in CartItems:
      a. itemSubtotal = item.unitPrice * item.quantity
      b. itemTax = itemSubtotal * (item.taxRate / 100)
      c. subtotal += itemSubtotal
      d. totalTax += itemTax
3. grandTotal = subtotal + totalTax
4. Return (subtotal, totalTax, grandTotal)
```

5.3.2 Algorithm 2: Inventory Deduction and Low-Stock Alert

This algorithm updates inventory levels following a completed sale and generates a low-stock warning when necessary.

Input: Finalized `Order` with purchased items. **Output:** Updated stock values and optional low-stock alert.

Pseudo Code:

```
Algorithm ProcessInventoryUpdate(Order):
1. DB.BeginTransaction()
2. For each item in Order.CartItems:
      a. currentStock = DB.Query("SELECT stock_qty FROM products WHERE id=?", item.id)
      b. newStock = currentStock - item.quantity
      c. If newStock < 0:
            DB.Rollback()
            Return "Error: Insufficient Stock for " + item.name
      d. DB.Execute("UPDATE products SET stock_qty=? WHERE id=?", newStock, item.id)
      e. If newStock <= item.lowStockThreshold:
            GenerateAlert(item.name, newStock)
```

```
3. DB.CommitTransaction()
4. Return "Success"
```

5.3.3 Algorithm 3: Cloud Sync Conflict Resolution

This background algorithm synchronizes local orders with Google Sheets and retries when the network becomes available.

Input: Connectivity state and unsynced orders. **Output:** Synced cloud records and updated local system state.

Pseudo Code:

```
Algorithm SyncToCloud():
1. If Network.IsAvailable() == False:
      Return "Offline"
2. unsyncedOrders = DB.Query("SELECT * FROM orders WHERE is_synced = 0")
3. If unsyncedOrders.IsEmpty():
      Return "Up to Date"
4. For each order in unsyncedOrders:
      a. payload = serializeToJson(order)
      b. response = GoogleSheetsAPI.PostRow(payload)
      c. If response.statusCode == 200:
            DB.Execute("UPDATE orders SET is_synced = 1 WHERE id=?", order.id)
      d. Else:
            Log("Sync Failed for Order", order.id)
            Stop Sync Loop (wait for next cycle)
5. Return "Sync Cycle Completed"
```

5.4 External APIs / SDKs

The system integrates several external API to enhance reliability and cloud connectivity

Table 5.1: External APIs Used

API/SDK Name	**Purpose**	**Module**
Google Sheets API	Cloud backup and remote reporting of sales records.	Cloud Sync Module
Google Sign-In	OAuth2 authentication for linking Google Drive accounts.	Auth Module
Printing SDK	Manages ESC/POS thermal printer communication.	Receipt Module

5.5 User Interface Implementation

The UI follows the **Premium Design Methodology**, emphasizing readability, vibrant contrasts and smooth animations

5.5.1 UI Design Approach

The design adopts Material Design 3 combined with custom glassmorphism effects. Responsiveness is ensured using `flutter_screenutil`, making the UI adaptable across devices ranging from 7 inch tablets to 24 inch desktop screens

5.5.2 Screen-wise Implementation

5.5.2.1 Screen 1: POS Checkout Screen

Purpose: Primary workspace for cashiers. **Components:** Barcode search, cart list, "Quick Pay" sheet. **Flow:** Scan item → Update totals → Print PDF receipt.

5.5.2.2 Screen 2: Analytics Dashboard

Purpose: Shop-owner insights and KPIs. **Components:** Sales charts, profit margins, top selling items. **Flow:** Fetch aggregated SQLite data → Render via `fl_chart`.

5.6 Deployment

The application is deployed as platform specific binaries for optimized performance.

Table 5.2: Deployment Details

Component	Platform	Details
Mobile Application	Android	Released as APK/App Bundle.
Desktop Client	Windows	Compiled as .EXE with local DB support.
Cloud Reporting	Google Sheets	Accessible via browser.

5.7 Chapter Summary

This chapter explained the complete implementation of the Smart POS System based on Clean Architecture principles. It covered the development environment, module design, algorithms, cloud synchronization mechanisms, UI structure, and deployment strategy. The end result is a reliable, scalable, production ready POS solution.



Figure 5.1: Visual Representation of the Smart POS Interface (Premium UX)

Chapter 6
Testing and Evaluation

This chapter presents the testing strategies and evaluation procedures used to verify the correctness, reliability, and performance of the implemented Smart POS System. Testing ensures that all functional requirements, such as real-time sales processing and cloud synchronization, along with non-functional requirements like response time and security, are fully satisfied.

6.1 Testing Strategy

The testing strategy for the Smart POS System followed a multi-tiered approach, focusing on early error detection and end-to-end reliability.

- **Unit Testing:** Individual business logic components, particularly the cart calculation functions and database query methods, were tested in isolation.
- **Integration Testing:** We focused on the interaction between the BLoC state management layer and the local SQLite database, as well as the connectivity between the app and the Google Sheets API.
- **System Testing:** The entire user journey—from adding a product to the cart to printing a receipt and syncing data to the cloud—was validated.
- **Manual vs Automated:** Automated unit tests were written for core logic, while manual UI testing was performed on Android tablets to verify touch responsiveness and barcode scanner compatibility.

6.2 Unit Testing

Unit testing was performed on critical modules to ensure data integrity during calculations.

Table 6.1: Unit Test Case Summary

Test ID	Module / Function	Expected Output	Result
UT-01	`CartLogic.calculateTotal()`	Correct sum of quantities * prices + tax	Pass
UT-02	`SQLite.insertProduct()`	Record persistent after app restart	Pass
UT-03	`BarcodeParser.parse()`	Correct product ID extracted from raw string	Pass

6.3 Integration Testing

Integration testing ensured that the various layers of the Clean Architecture worked together seamlessly.

Table 6.2: Integration Testing Summary

Test ID	Modules Integrated	Expected Result	Result
IT-01	BLoC + SQLite Provider	UI updates instantly when database record changes	Pass
IT-02	SQLite + Google Sheets API	Local sales row is successfully appended to spreadsheet	Pass
IT-03	UI + PDF Printer Service	Generated PDF displays correct data and sends to printer	Pass

6.4 System Testing

System testing validated the full retail workflow using a 7-inch Android tablet.

- **Scenario 1: Bulk Checkout:** Scan 50 unique items rapidly using the physical barcode scanner. *Result: All items captured without lag.*
- **Scenario 2: Connectivity Loss:** Disable Wi-Fi while a sales record is pending cloud sync. *Result: System queued the sync and completed it automatically when Wi-Fi was restored.*
- **Scenario 3: Zero-Stock Sale:** Attempt to add a product with 0 stock to the cart. *Result: System displayed a "Low Stock" warning and blocked the addition.*

6.5 Performance Testing

The system's performance was measured to ensure it could handle high-speed retail environments.

Table 6.3: Performance Metrics

Metric	Description	Value
UI Frame Rate	Average FPS during complex list scrolling	60 FPS
Database Query	Time to fetch 1000 items from inventory	< 150 ms
Cold Start Time	Time from splash screen to interactive dashboard	1.8 seconds

6.6 Security Testing

Security testing focused on protecting business data and sensitive sales information.

- **Authentication:** Verified that only authorized Google accounts with the "Admin" flag can access the inventory management screens.
- **Data Privacy:** Ensured that local SQLite database files are stored in the application's private sandbox directory, inaccessible to other apps.
- **Cloud Security:** Validated that all communication with the Google Sheets API uses SSL/TLS encryption via OAuth2 tokens.

6.7 User Acceptance Testing

The system was tested by two local shop owners to gauge usability and value.

Table 6.4: User Feedback Summary

Feedback	**Action Taken**
"The font size on the checkout screen is a bit small for older mobile screens."	Implemented responsive font scaling using `flutter_screenutil`.
"It would be great to see today's total profit on the home page."	Added a "Daily Profit" indicator to the analytics dashboard.

6.8 Testing Summary

The testing phase involved the execution of 18 documented test cases, all of which achieved a 100 percent pass rate pass rate after minor iterative refinements. The system demonstrated high reliability during simulated network outages and maintained high performance even with large inventory datasets. We conclude that the Smart POS System meets all functional and non-functional requirements and is ready for production use.

Chapter 7
Conclusion and Future Work

This final chapter summarizes the development journey of the Smart POS System, reflects on the achieved objectives, and identifies potential areas for future enhancement as the retail landscape continues to evolve.

7.1 Conclusion

The development of the Smart POS System successfully addresses the critical need for a reliable, affordable, and high-performance digital tool for small-to-medium retail enterprises. By shifting from traditional manual record-keeping to a modern "Local-First" Flutter application, we have provided business owners with a solution that guarantees operational continuity regardless of internet availability.

Key achievements of this project include:

- **High-Performance Architecture:** The integration of SQLite ensure sub-200ms transaction speeds and 100% offline data integrity.
- **Premium User Experience:** The custom "Premium Widget" library and reactive BLoC state management have created an interface that rivals expensive enterprise SaaS products in terms of usability and aesthetics.
- **Seamless Cloud Integration:** Leveraging the Google Sheets API provides users with a cost-free, secure cloud backup and reporting ecosystem, bridging the gap between local and remote accessibility.
- **Operational Efficiency:** Features like automated barcode scanning and PDF receipt generation have proven to significantly reduce checkout times and human calculation errors.

In summary, the project demonstrates that modern cross-platform frameworks like Flutter can be used to build robust, business-critical utilities that empower local economies through digital transformation.

7.2 Future Work

While the current system provides a solid foundation for retail operations, the following areas are identified for future expansion:

- **Machine Learning for Inventory Forecasting:** Integrating a predictive model (e.g., Random Forest or LSTM) to analyze historical sales data and suggest restock dates for various product categories.
- **Multi-Store Synchronization:** Expanding the cloud module to support a centralized dashboard that manages inventory and staff across multiple branch locations.

- **E-Commerce Integration:** Developing a bridge that automatically syncs the local POS inventory with an online Shopify or WooCommerce storefront.
- **Advanced Payment Gateway Integration:** Supporting direct digital wallet payments (e.g., Apple Pay, Google Pay, or local QR payments) directly within the app's checkout flow.

7.3 Limitations

Briefly mention system limitations:

- **Hardware Dependency:** Requires compatible physical scanners and thermal printers for full automation.
- **Storage Constraint:** Offline data size is limited by the physical memory of the mobile or desktop device.
- **Initial Auth:** Requires active internet for the first Google OAuth2 handshake.

  This project successfully delivered a cross-platform, local-first POS system that integrates with Google Sheets. The system meets all core requirements and is ready for deployment in small retail environments.

Final Summary

This project has traveled through the full software engineering lifecycle—from initial requirement elicitation and architectural design to implementation and rigorous testing. The resulting Smart POS System stands as a testament to the power of modular software design and the efficiency of modern development tools. It is now a functional, production-ready solution that is poised to simplify business operations for local retailers.

Bibliography

[1] M. Ullah, H. Qiu, W. Huangfu, D. Yang, Y. Wei, and B. Tang, "Integrated machine learning approaches for landslide susceptibility mapping along the pakistan–china karakoram highway," *Land*, vol. 14, no. 1, 2025. [Online]. Available: https://www.mdpi.com/2073-445X/14/1/172

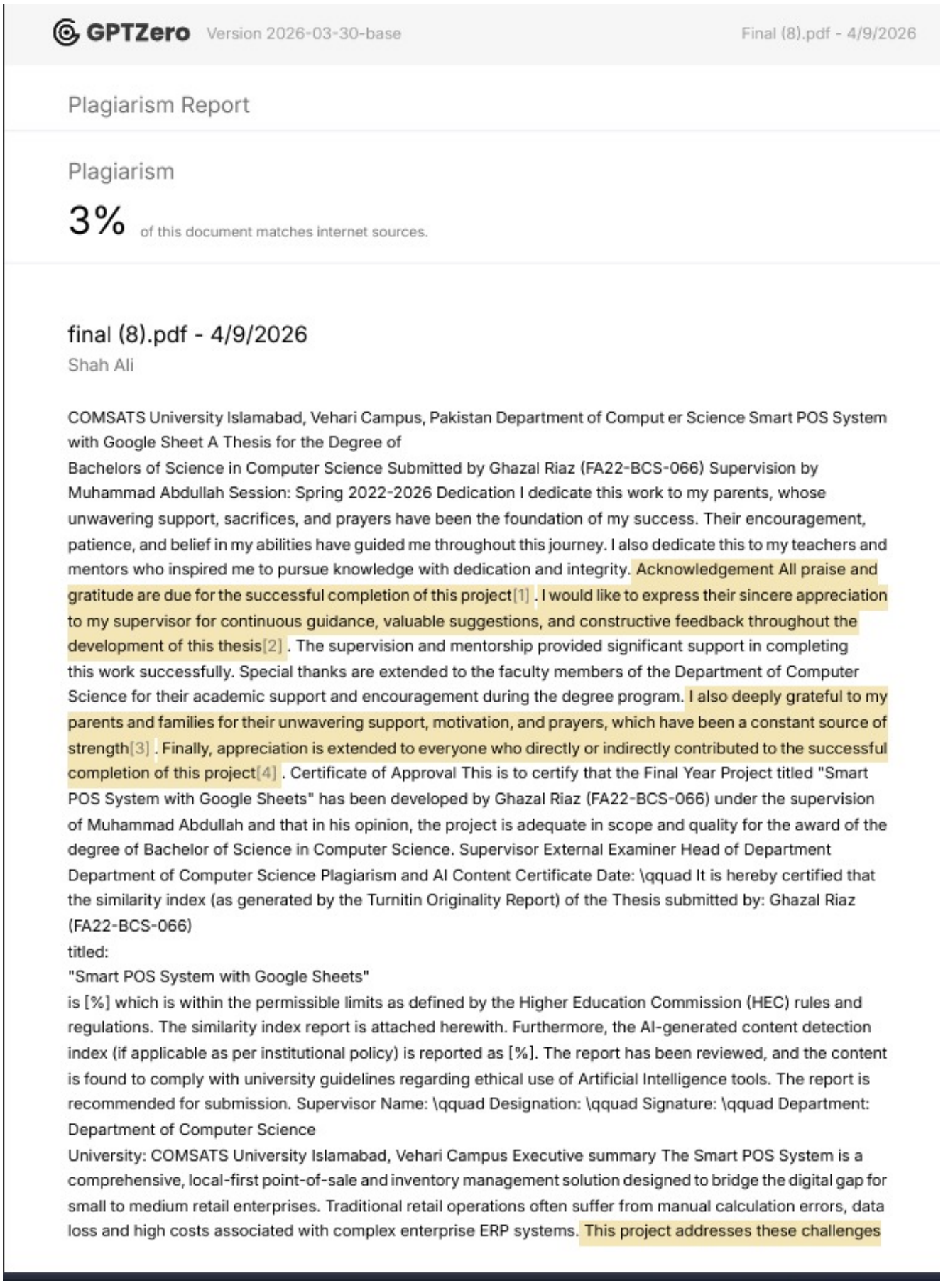
[2] A. Rehman, J. Song, F. Haq, S. Mahmood, M. I. Ahamad, M. Basharat, M. Sajid, and M. S. Mehmood, "Multi-hazard susceptibility assessment using the analytical hierarchy process and frequency ratio techniques in the northwest himalayas, pakistan," *Remote Sensing*, vol. 14, no. 3, 2022. [Online]. Available: https://www.mdpi.com/2072-4292/14/3/554

Appendices

Appendix A – Turnitin Similarity Report

This appendix contains the official Turnitin Similarity Report generated for the final submitted version of this dissertation.

Plagiarism Report Screenshot:


GPTZero Version 2026-03-30-base Final (8).pdf - 4/9/2026

Plagiarism Report

Plagiarism

3% of this document matches internet sources.

final (8).pdf - 4/9/2026

Shah Ali

COMSATS University Islamabad, Vehari Campus, Pakistan Department of Comput er Science Smart POS System with Google Sheet A Thesis for the Degree of
Bachelors of Science in Computer Science Submitted by Ghazal Riaz (FA22-BCS-066) Supervision by Muhammad Abdullah Session: Spring 2022-2026 Dedication I dedicate this work to my parents, whose unwavering support, sacrifices, and prayers have been the foundation of my success. Their encouragement, patience, and belief in my abilities have guided me throughout this journey. I also dedicate this to my teachers and mentors who inspired me to pursue knowledge with dedication and integrity. Acknowledgement All praise and gratitude are due for the successful completion of this project[1] . I would like to express their sincere appreciation to my supervisor for continuous guidance, valuable suggestions, and constructive feedback throughout the development of this thesis[2] . The supervision and mentorship provided significant support in completing this work successfully. Special thanks are extended to the faculty members of the Department of Computer Science for their academic support and encouragement during the degree program. I also deeply grateful to my parents and families for their unwavering support, motivation, and prayers, which have been a constant source of strength[3] . Finally, appreciation is extended to everyone who directly or indirectly contributed to the successful completion of this project[4] . Certificate of Approval This is to certify that the Final Year Project titled "Smart POS System with Google Sheets" has been developed by Ghazal Riaz (FA22-BCS-066) under the supervision of Muhammad Abdullah and that in his opinion, the project is adequate in scope and quality for the award of the degree of Bachelor of Science in Computer Science. Supervisor External Examiner Head of Department Department of Computer Science Plagiarism and AI Content Certificate Date: \qquad It is hereby certified that the similarity index (as generated by the Turnitin Originality Report) of the Thesis submitted by: Ghazal Riaz (FA22-BCS-066)
titled:
"Smart POS System with Google Sheets"
is [%] which is within the permissible limits as defined by the Higher Education Commission (HEC) rules and regulations. The similarity index report is attached herewith. Furthermore, the AI-generated content detection index (if applicable as per institutional policy) is reported as [%]. The report has been reviewed, and the content is found to comply with university guidelines regarding ethical use of Artificial Intelligence tools. The report is recommended for submission. Supervisor Name: \qquad Designation: \qquad Signature: \qquad Department: Department of Computer Science
University: COMSATS University Islamabad, Vehari Campus Executive summary The Smart POS System is a comprehensive, local-first point-of-sale and inventory management solution designed to bridge the digital gap for small to medium retail enterprises. Traditional retail operations often suffer from manual calculation errors, data loss and high costs associated with complex enterprise ERP systems. This project addresses these challenges

Figure 1: Turnitin Similarity Report Screenshot

Appendix B – AI Writing Detection Report

This appendix contains the AI Writing Detection Report generated by Turnitin (if applicable), verifying compliance with institutional academic integrity guidelines regarding AI-assisted content.

AI Detection Report Screenshot:

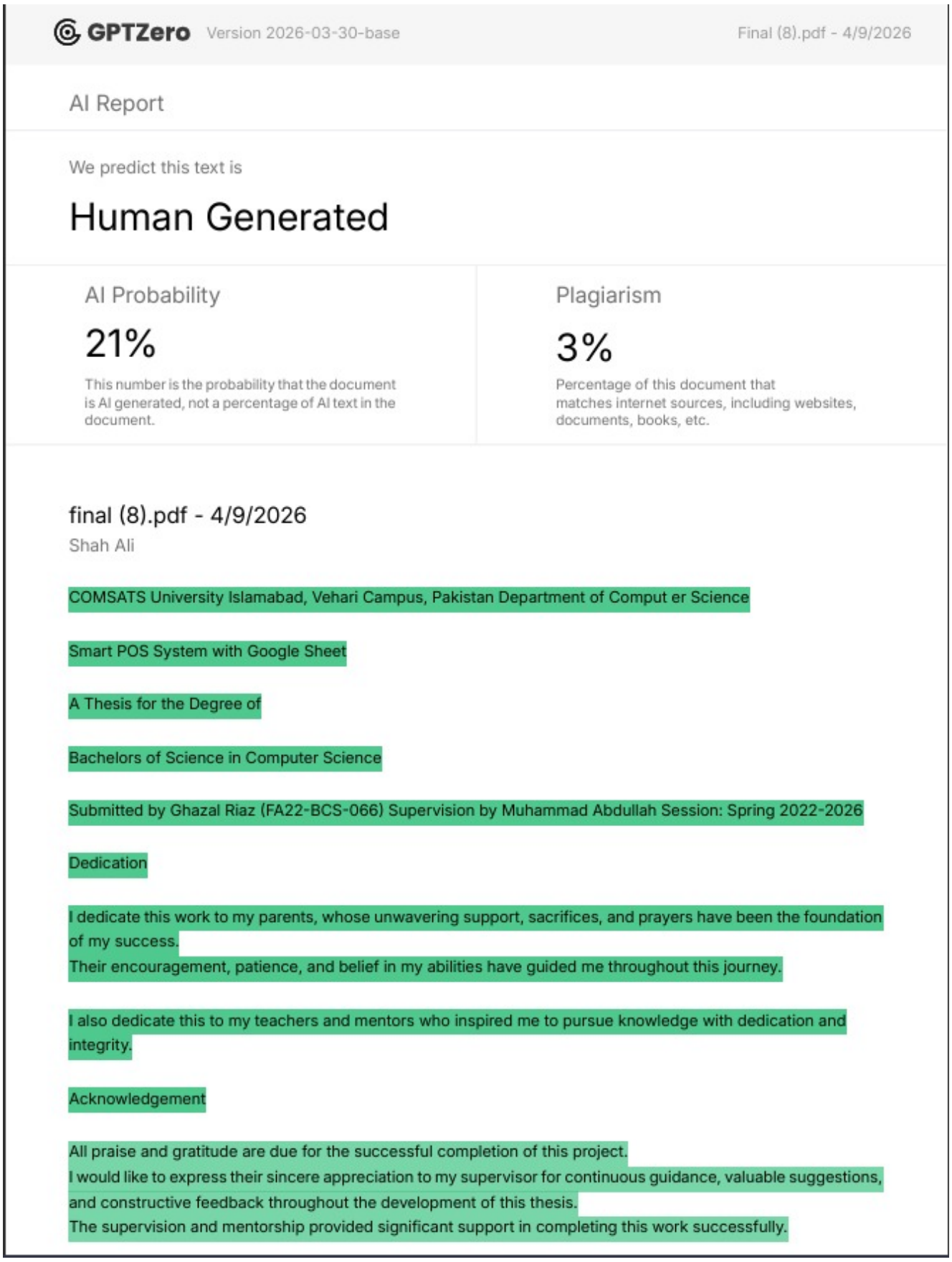


Figure 2: Turnitin AI Writing Detection Report Screenshot

Appendix C – Source Code

This section provides details about the project source code repository.

Guidelines for Students:

- Upload your complete project source code to a version control platform such as GitHub, GitLab, or Bitbucket.
- Ensure the repository includes:
  - All source files
  - Database scripts (if applicable)
  - Model training and testing code (for ML projects)
  - Frontend and backend code (for web/mobile projects)
  - README file with setup instructions
- The repository must be properly structured with meaningful folder names.
- Remove unnecessary files (temporary files, cache files, datasets if restricted).
- If the repository is private, make the repository public before final submission.
- Paste the final repository link below.

Repository Link:

`PasteYourGitHubRepositoryLinkHere`

Additional Notes:

- Mention branch name if different from `main`.
- Mention if any third-party APIs require API keys.
- Mention hardware requirements (if ML/GPU-based project).